Milestone-C Repeat Test Execution and Evidence Finalisation

Melbourne Urban Forest API checks

1 Document details

Field	Value
Prepared by	Darvin Anorma Mangampo
Assessment period	15–19 October 2026 (UTC+08:00)
Test execution dates	15–16 October 2026 (UTC+08:00)
Repeat run date	16 October 2026 (UTC+08:00)
Repository commit present during repeat run	1ed2bc338de6895b618aa82ca5cede3fc4a36e98
Document date	19 October 2026

2 Purpose and testing context

After completing The Test Tribe Playwright course, I began this API testing project in 2026 as part of my upskilling. I used Melbourne's public Urban Forest API to apply those skills to live responses. The project is separate from my previous role, and no client requirements were supplied. I used the same eight T01–T08 conditions that I had implemented for Milestone-B. In the test matrix, I described what I expected to see in the API responses for each condition. I based those expectations on the City's published API rules, the Milestone-A observations and comparisons made within each run.

The Test Tribe Playwright course covered GET requests through the `request` fixture, checking JSON responses with `expect`, and reading test data from JSON. I used those techniques to build the Urban Forest checks. I chose the requests and expected results from the City's API guidance and the responses I observed.

Milestone-B saved the first complete T01–T08 result on 15 October. In my test plan, I set out a second complete execution on another calendar date. I ran the eight conditions again on 16 October because the 15 October result showed only how the API responded during that run. If a later result had differed from its matrix expectation, I would have examined the request, response and current data before interpreting the difference. The two saved runs let me compare the assertions and returned values on different dates. The two runs sampled the API during short periods on 15 and 16 October. Because I did not monitor the service throughout the time between them, their passing results cannot show whether it was available continuously. I did not choose a particular City API update to assess. I repeated the eight checks set out in my test plan because I wanted to see whether they still met their expected results on a later date. The ISTQB Foundation Level syllabus describes regression testing as checking for adverse effects caused by a change. Because this comparison did not assess the effect of an identified change, I describe it as a repeat execution.

3 Execution approach

The test specification, JSON boundary cases, Playwright configuration, package file and lockfile have matching saved SHA-256 hashes in both runs. The matching hashes show that my five test files had the same content for both runs. They do not describe Melbourne's tree data because the hashes were calculated from my files. I used the suite to check selected API responses during two short runs. Because it did not inspect every tree entry or monitor the dataset between dates, it cannot establish that the entire dataset stayed unchanged. That limit fits these eight conditions because they check API responses to a small set of consumer requests. Commit 1ed2bc338de6895b618aa82ca5cede3fc4a36e98 was already present when the 16 October run began. I configured Playwright to use one worker and turned off parallel execution because I wanted to keep simultaneous traffic to the public API low. I also set retries to zero because an automatic second attempt could conceal a failure from the first. The test code paused 700 ms before each GET request and stopped on an access-limit response.

All eight matrix conditions ran. T05 ran two cases because it checked limit=100 and limit=101 separately. T06 ran two more because it checked an unknown field in where and then in order_by. Each of the other six conditions ran once. The complete run therefore had ten Playwright test cases across the eight conditions. When I ran the suite on 16 October, Playwright sent 13 GET requests. In the test matrix, I had set an expectation of fewer than 20 requests for a full run. I used GET because the Explore API v2.1 reference says it is the only supported HTTP method. I did not request an export because these eight conditions checked small tree responses rather than the full dataset. None of the requests changed the City's data.

I ran the 16 October suite from the repository root and saved the actual command and tool versions. The HTML report shows the result of each of the ten Playwright cases. The run summary keeps the command and overall result. The detailed test evidence saves the requests, responses and values checked against the matrix.

4 Comparison of the two complete runs

Item	15 October baseline	16 October repeat run
Outcome	10 passed, none failed or skipped	10 passed, none failed or skipped
GET requests	13	13
HTTP responses	Ten HTTP 200 and three expected 400	Ten HTTP 200 and three expected 400
Tree totals	82,064 in data and metadata	82,064 in data and metadata
Source identity	Source hashes saved before the Milestone-B source commit	Same five source hashes. Commit 1ed2bc3 already present

I compared the saved evidence for all 13 requests. Their paths and response summaries matched, including the statuses and selected values. The sampled behaviour met the matrix expectations again on 16 October. I ran the suite for a short period on 15 October and again on 16 October. Each execution ended when its checks finished. I did not run the suite as a continuous monitor between those dates. Because it collected responses only during the two executions, the matching results describe how the API answered at those times. They cannot establish whether the tree data or the service stayed the same throughout the time between them.

5 Results of the repeat run by matrix condition

Check	Observed result on 16 October
T01	HTTP 200 JSON with five rows and a numeric total of 82,064. The saved sample has com_id 1023218 and genus Platanus. This met T01 because the response contained tree rows and the populated fields needed for a usable sample.
T02	The test requested two five-row pages in com_id order. It checked that all ten IDs were in ascending text order without a duplicate. Both ordered lists and ID sets are saved. This met T02 because a consumer could reach a populated second page without receiving an ID already present on the first.
T03	The lookup returned the same com_id 1013381 and genus Ulmus as the source row. This met T03 because the tree selected from a current response could be found again with those same two values.
T04	The filter returned five rows, and every saved genus value was Ulmus. This met T04 because the filter returned trees and none of the returned rows had a different genus.
T05	The published maximum, limit=100, returned 100 rows with HTTP 200. One above that maximum, limit=101, returned HTTP 400 JSON with a limit error. This met the boundary pair because the API accepted the published maximum and explained why it rejected the next page size.
T06	An unknown field in where returned HTTP 400 JSON. A separate order_by request did too. Each error message identified its field and clause. This met T06 because a caller could identify which part of each query was invalid from the response.
T07	The unfiltered data and metadata totals both read 82,064. The observation time is saved with the values. This met the comparison rule because the two API views agreed during this run. It was not a separate count of physical trees.
T08	The unused com_id returned HTTP 200, total_count 0 and an empty results array. This met T08 because the lookup gave an explicit empty result for a missing tree instead of returning a tree or causing the test to fail unexpectedly.

6 Interpretation and limits

All eight matrix conditions met their expectations on both dates. All ten Playwright test cases passed on each date. The T05 and T06 input pairs explain why each complete run had ten cases for eight conditions, as described in section 3. On 16 October, the second sorted page again held five trees and the genus filter returned five Ulmus rows. The detailed evidence saves the sampled fields, both ordered ID lists and sets, every compared genus, the two totals and the error responses required by the matrix. The results show what the service returned on those dates. The passing results show that these requests worked during the two short runs. They cannot establish continuous availability because the suite did not make requests throughout the time between them. An interruption during that interval would not appear in either saved result.

Both the 15 and 16 October runs had ten Playwright cases and sent 13 GET requests. T01 used two requests because setup first fetched a current tree and the check then sampled another five-row response. T02 fetched two pages. T07 fetched data and metadata separately. The other seven cases made one request each. The Platanus row saved for T01 is different from the Ulmus tree selected during setup for T03 and T04. Both are current API values, not conflicting identities. Both runs had the same case and request counts.

The data response and metadata each reported 82,064 trees on 16 October. These are two views of the same published dataset, so their agreement checks the API responses against each other rather than against a separate physical inventory. I checked small API responses because the eight conditions concern how a consumer retrieves and queries tree data. They did not require an export of every published entry. I also did not compare the API data with a separate tree inventory because checking the completeness and accuracy of the physical inventory was outside this test plan. The matching API totals therefore show agreement between two responses from the same dataset. They cannot establish whether every tree is represented or whether each entry is correct.

During the Milestone-A preflight, I ran a Playwright script to request a page at the deep-paging boundary. With limit=100 and offset=9900, the API returned HTTP 200 and 100 rows. When I increased the offset to 9901, it returned HTTP 400 with a message saying the sum must be no greater than 10,000. The published Explore API reference says the sum should be less than 10,000. The Milestone-A request tested deep paging at offset + limit = 10,000. I did not include that boundary among T01–T08 because this suite used the first two small pages for T02 and the separate page-size limit of 100 and 101 for T05. The published rule and the Milestone-A response describe the deep-paging boundary differently. No 16 October assertion failed because none of the eight conditions made that deep-paging request.

Neither complete run sent a request at offset + limit = 10,000. Because the suite did not ask the API about that boundary on 15 or 16 October, its passing cases cannot show whether the 13 October response would occur again. The 15 and 16 October runs show how the API answered T01–T08 on those dates. I keep the 13 October boundary response as a separate dated observation.

7 Supporting evidence

The Milestone-B baseline summary and Milestone-C run summary save the actual commands, source identities and outcomes. Each HTML report shows ten Playwright case results for the eight matrix conditions because T05 and T06 each used two inputs. The other six conditions ran once. The detailed test evidence retains the response values behind the assertions, including both ordered ID lists for T02, every compared genus for T04 and the two T07 totals. The shared run summary compares the dates and explains why the source commits differ even though the five tested files have matching hashes. The Milestone-C run summary gives the command used on 16 October. The HTML report and detailed test evidence show its outcomes and checked values.

Evidence	Purpose
Test specification and matrix tests/urban-forest.api.spec.ts and evidence/test-matrix.md	The executable checks and their expected-result basis.
Milestone-B baseline baseline-run-summary.json and html-report/index.html	The 15 October command, source hashes and ten test outcomes.
Milestone-C README README.md	Run context, reproduction steps and links to the saved result.
Milestone-C run summary 16 October summary JSON	The 16 October command, versions, source hashes, existing commit and test outcomes.
Milestone-C test evidence test-evidence.json	Requests, responses and values checked for T01–T08.
Milestone-C HTML report html-report/index.html	Individual Playwright outcomes from 16 October.
Project comparison and findings run-summary.md and findings.md	Comparison of the two dates and limits of their observations.

In the test plan, I set out eight matrix conditions to run on two different dates. Each complete run produced ten Playwright case results because T05 and T06 each had two inputs. Both runs covered all eight conditions. Their detailed evidence retains the values requested by the matrix. The 15 October summary identifies its tested files by hash and has no commit ID at execution. Commit 92c23d4c9fe7b81abb8a05eef51a6dcbc88805b0 later added that source and baseline evidence. The 16 October summary names the already existing commit 1ed2bc338de6895b618aa82ca5cede3fc4a36e98. The baseline remains a 15 October execution because the requests were made that day and its summary saved the source hashes. The later commit places those tested files and the baseline evidence in the repository history.

The City dataset page supplied the field meanings used for the tree checks. The Explore API reference supplied rules for page size, query fields and deep paging. Those published statements form the P basis in the matrix. The observed 13 October responses form its O basis, while comparisons between responses within a run form R. I used ISTQB Foundation Level section 1.4.1 as a supplementary reference when reviewing the test activities and saved results. I also used section 5.1.3 when checking whether I had met the completion criteria in my test plan. The City documentation and actual API responses remain the basis for these test expectations. I also consulted section 2.2.3 for the distinction between regression testing and the repeat execution described in section 2.

8 Public access to the evidence

On 19 October 2026, I checked the earlier evidence that was publicly available on GitHub. I opened the Milestone-A README and PDF report and followed their links. I did the same for Milestone-B. I then checked every link in the published test plan, test matrix and findings note. Each destination opened when I checked it.

I made this check because the comparison in this report relies on the earlier test expectations and the 15 October result. I wanted to confirm that the links I used led to the files and sources behind those claims. GitHub opened the saved Milestone-B HTML report as a file page. It displayed the HTML source rather than an interactive Playwright report.

Before uploading the Milestone-C files, I checked the links in this report and the accompanying documents against their intended repository paths. The links are designed to work on GitHub once those files are uploaded.

9 Conclusion

On 16 October, I ran all eight matrix conditions. All ten Playwright test cases passed on 16 October. Together, they covered the eight matrix conditions described in this report. The 15 October baseline also shows ten passes using the same five source files. The checks reached a populated second page, matched a current tree in a lookup, rejected the intended invalid inputs and found equal data and metadata totals. These results describe selected API requests on two dates. They cannot establish the accuracy of every tree entry because I did not compare the published list with a separate tree inventory. They cannot establish uninterrupted availability because the suite made no requests throughout the time between the runs. Neither run checked offset + limit = 10,000 because T02 used the first two small pages and T05 checked page sizes 100 and 101. The Milestone-A deep-paging observation remains a separate dated question.